Submission Worksheet

IT202-450-M2026 / module-06-worksheet-multi-dimensional-php-arrays

Submission Data

Course	IT202-450-M2026
Assignment	Module 06 Worksheet: Multi-Dimensional PHP Arrays
Student	Shakir A. (sa3327)
Status	submitted
Worksheet Progress	100%
Potential Grade	NaN/NaN (NaN%)
Updated	7/14/2026, 11:50:56 AM

Grading Link
https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/module-06-worksheet-multi-dimensional-php-arrays/grading/sa3327

View Link
https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/module-06-worksheet-multi-dimensional-php-arrays/sa3327

Instructions

Complete the Module 6 multi-dimensional PHP array problems in the homework branch, then collect focused evidence for each problem. It is best to solve and test the PHP files first, then fill out the worksheet; however, you may also collect evidence as each problem is solved.

Assignment Setup

- Starter reference: Module 6 Homework branch.
- Required branch: `Module06-Homework`
- Required starting branch: `qa`
- Required homework folder: `public_html/m06/hw/`
- Starter files to copy:
  - `base.php`
  - `index.php`
  - `problem1.php`
  - `problem2.php`
  - `problem3.php`
- Recommended order:
  - Read the whole worksheet.
  - Copy the starter files; commit the baseline.
  - Add UCID/date and planning comments where edits are allowed.
  - Commit the planning-comment stage before solving the problems.
  - Solve each problem, test through `qa`, then collect the worksheet evidence.

Git Workflow Checklist

1. Check out `qa`.
2. Pull the latest `qa` branch with `git pull origin qa`.
3. Create the homework branch with `git checkout -b Module06-Homework`.

2. Pull the latest `qa` branch with `git pull origin qa`.
3. Create the homework branch with `git checkout -b Module06-Homework`.
4. Copy the starter files into `public_html/m06/hw/`.
5. Commit the baseline starter files before solving the problems.
6. Push the branch with `git push origin Module06-Homework`.
7. Open a pull request from `Module06-Homework` into `qa` and keep it open while you work.
8. Add planning comments before the final code for every problem.
9. Commit planning comments before the finished solution for each problem.
10. Commit working code in small pieces as each problem starts working.
11. After all problems pass locally, merge the homework pull request into `qa`.
12. Gather evidence from the deployed `qa` site and fill out the worksheet.
13. Export the worksheet PDF from the Learn Courses Platform.
14. Save the PDF in the repository under `docs/module06/`.
15. Add, commit, and push the PDF to the `Module06-Homework` branch.

    - `git add .`
    - `git commit -m "Add module 6 worksheet PDF"`
    - `git push origin Module06-Homework`

16. Upload the same PDF to Canvas.
17. Create and merge a pull request from `qa` into `prod` so the anticipated production URLs become live.
18. Switch back to `qa` with `git checkout qa`.
19. Pull the latest `qa` branch with `git pull origin qa`.

Shared Requirements

Apply these requirements to each problem:

- Only edit code where the starter comments say edits are allowed.
- Add a comment with your UCID, date, and a brief summary of the solution approach.
- Add planning comments before the solution code.
- Commit the planning-comment stage before the final solution.
- Solve inside the provided `Start Solution Edits` and `End Solution Edits` section.
- Use the function parameters, not the specific `$a1`, `$a2`, `$a3`, or `$a4` variables, inside the solution logic.
- Keep the provided output helpers, navigation, and external validation hooks in place.
- Test the file through the supported server path before collecting output evidence. For deployed evidence, use the working `qa` URL after the site has deployed.
- Use your own code, output, branch, pull request, and deployed URLs for evidence.

Problem Evidence Format

Each problem uses the same evidence pattern:

- Images:
  - Show the relevant code and the matching browser output.
  - Make sure the UCID/date comment is visible in the code evidence.
  - Show enough output to prove all four provided data sets were processed.
  - One combined screenshot is acceptable if the code and output are both readable.
  - Use two screenshots when one image would be hard to read.
- URLs:
  - Provide the direct GitHub file link from the homework branch.
  - Provide the anticipated production URL for the PHP file.
  - Capture the working `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
  - Each file URL must end in `.php`; zero credit for that URL entry if it does not.
- Response:

- Capture the working `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
- Each file URL must end in `.php`; zero credit for that URL entry if it does not.

- Response:
  - Explain the logic in your own words.
  - Describe the steps or decisions your PHP makes.
  - Do not restate the prompt or list the requirements.

Submission Check

- Confirm the homework branch has been pushed.
- Confirm the pull request into `qa` exists.
- Confirm the baseline, planning, and solution commits are visible.
- Confirm the `qa` version was reviewed before promotion.
- Confirm the anticipated production URLs use the same path as the tested `qa` URLs with `-qa` changed to `-prod`.
- Confirm the worksheet PDF is committed under `docs/module06/` on the homework branch.
- Upload the same worksheet PDF to Canvas.

Submission Progress 100%

Section #1: (3 pts.) Problem 1 Subset

100%

- File to edit and test: `public_html/m06/hw/problem1.php`
- Goal: edit the `processBirds()` function so it extracts each bird's `name`, `color`, and `region` into `$subset`.
- Expected behavior: `$subset` should be a multi-dimensional array with one row per bird.
- Expected fields: each output row should include only `name`, `color`, and `region`.
- Starter rule: the function should loop over the `$birds` parameter and should not reference `$a1`, `$a2`, `$a3`, or `$a4` directly.

Task #1 (3 pts.) – Problem 1 Evidence

100%

Combo #1

Part 1: Images 100%

ⓘ **Details**

- Show the relevant code with the UCID/date comment visible.
- Show the full browser output after running the file.

```
// se3327 6/14
// plan: loop through $birds arr
// for each bird, create a new arr with only name color and region
// add that arr to $subset

$subset = [];
// Start Solution Edits
foreach ($birds as $bird) {
    $subset[] = [
        "name" => $bird["name"],
        "color" => $bird["color"],
        "region" => $bird["region"]
    ];
}
// End Solution Edits
```

code

browser

Part 2: URLs 100%

ⓘ **Details**

- Paste the direct GitHub file link from the homework branch.
  - The GitHub URL must end in `.php`; zero credit for this URL if it does not.
- Paste the anticipated production URL for the PHP file.
  - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
  - The production URL must end in `.php`; zero credit for this URL if it does not.

- Paste the anticipated production URL for the PHP file.
  - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
  - The production URL must end in `.php`; zero credit for this URL if it does not.

URL #1
https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Module06-Homework/public_html/m06/hw/problem1.php 👍

URL #2
https://sa3327-it202-450-m2026-prod.onrender.com/m06/hw/problem1.php 👍

Part 3: Response 100%

Details

- Explain how your loop builds `$subset`.
- Explain how your solution chooses only the required keys.
- Explain why it works for all four bird data sets.
- Do not restate the prompt.

Response

The loop goes through each bird and adds its name, color, and region to $subset. It only accesses the speific keys it needs. It works for all four data sets because it uses the $birds parameter instead of a specific array.

Supports **bold**, *italic*, [links](url), `code`, etc. 222 chars

Section #2: (3 pts.) Problem 2 Adding Properties

100%

- File to edit and test: `public_html/m06/hw/problem2.php`
- Goal: edit the `processCars()` function so it creates `$processedCars` with the original car properties plus `age` and `isClassic`.
- Expected behavior: each output row should keep the original `id`, `make`, `model`, and `year` values.
- Expected derived values:
  - `age` is calculated from the current year and the car's year.
  - `isClassic` is a boolean based on the current year, the car's year, and `$classic_age`.
- Starter rule: the function should loop over the `$cars` parameter and should not reference `$a1`, `$a2`, `$a3`, or `$a4` directly.

Task #1 (3 pts.) – Problem 2 Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the relevant code with the UCID/date comment visible.
- Show the full browser output after running the file.

code

browser

code

browser

Part 2: URLs 100%

Details

- Paste the direct GitHub file link from the homework branch.
  - The GitHub URL must end in `.php`; zero credit for this URL if it does not.
- Paste the anticipated production URL for the PHP file.
  - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
  - The production URL must end in `.php`; zero credit for this URL if it does not.

URL #1
https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Module06-Homework/public_html/m06/hw/problem2.php

URL #2
https://sa3327-it202-450-m2026-prod.onrender.com/m06/hw/problem2.php

Part 3: Response 100%

Details

- Explain how your solution calculates age.
- Explain how your solution decides whether `isClassic` is true or false.
- Explain how your solution keeps the original car properties.
- Do not restate the prompt.

Response

Age is calculated as the current year minus the year of the car, and isClassic will be true if the age is greater than or equal to 25 and otherwise will be false, and the old car details were copied to the new array before new value details added.

Supports **bold**, *italic*, [links](url), `code`, etc. 247 chars

Section #3: (3 pts.) Problem 3 Join

100%

- File to edit and test: `public_html/m06/hw/problem3.php`
- Goal: edit the `joinArrays()` function so it combines user data and activity data by matching `userId`.
- Expected behavior: `$joined` should contain one row per matched user and activity pair.
- Expected fields: each joined row should include useful values from both the user row and the matching activity row.
- Starter rule: join by the shared `userId` key, not by hard-coded array names or assumptions about matching array positions.

Task #1 (3 pts.) – Problem 3 Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the relevant code with the UCID/date comment visible.
- Show the full browser output after running the file.

- Show the relevant code with the UCID/date comment visible.
- Show the full browser output after running the file.

code

browser

Part 2: URLs 100%

Details

- Paste the direct GitHub file link from the homework branch.
  - The GitHub URL must end in `.php`; zero credit for this URL if it does not.
- Paste the anticipated production URL for the PHP file.
  - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
  - The production URL must end in `.php`; zero credit for this URL if it does not.

URL #1
https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Module06-Homework/public_html/m06/hw/problem3.php

URL #2
https://sa3327-it202-450-m2026-prod.onrender.com/m06/hw/problem3.php

Part 3: Response 100%

Details

- Explain how your solution finds matching `userId` values.
- Explain how your solution combines data from both arrays.
- Explain why joining by key is safer than assuming matching array positions.
- Do not restate the prompt.

Response

The solution joins the userId from every user to the userId from every activity only when they are the same. The result is a new array, which has values from both arrays and by matching through userId, it is more secure because the arrays might not have the same order.

Supports **bold**, *italic*, [links](url), `code`, etc. 269 chars

Section #4: (1 pt.) Misc

67%

Task #1 (0.33 pts.) – GitHub Details

100%

Combo #1

Part 1: Images 100%

Details

- Screenshot the Commits tab of the pull request.

ⓘ Details

- Screenshot the Commits tab of the pull request.
- The screenshot should show at least the baseline commit, planning-comment commits, and solution commits.

commits

Part 2: URLs 100%

ⓘ Details

- Paste the pull request URL.
  - The URL must end in `/pull/#`, where `#` is the pull request number; zero credit for this URL if it does not.

URL #1
https://github.com/shakirahmed3317/sa3327-it202-450-m2026/pull/54/ 👍

Task #2 (0.33 pts.) – WakaTime Activity

100%

ⓘ Details

- Visit the WakaTime dashboard.
- Open `Projects`.
- Find your repository.
- Screenshot the overall time near the repository name.
- Screenshot the file-level activity.
- The purpose is evidence of activity; the duration is not tied to the grade.
- The visual graphs are not required.

waka

waka files

Task #3 (0.33 pts.) – Reflection

ⓘ Details

- Answer each question with at least a few reasonable sentences.

Task #1 (0.11 pts.) – What did you learn during this assignment?

- Answer each question with at least a few reasonable sentences.

Task #1 (0.11 pts.) – What did you learn during this assignment?

100%

Response

I learned how to traverse mutli-dimensional arrays and extract data out of them.

Supports **bold**, *italic*, [links](url), `code`, etc. 80 chars

Task #2 (0.11 pts.) – What was the easiest part of the assignment?

100%

Response

The easiest part was problem 1 where I just had to extract data from the array.

Supports **bold**, *italic*, [links](url), `code`, etc. 79 chars

Task #3 (0.11 pts.) – What was the hardest part of the assignment?

100%

Response

There wasn't really a hard part of the assignment. All the problems were mostly the same where you had to traverse the arrays but differed in the output data.

Supports **bold**, *italic*, [links](url), `code`, etc. 159 chars

End of Assignment - submission has been recorded.